Set 2: Student academic advising KRR system

Knowledge base, production rules, forward chaining, decision support

Irene Miranda
BSCS 3B

FINAL FILE:
CSST101 SET 2

Problem Scenario: Build a system that uses fictional student attendance and grade information to infer possible academic support recommendations.

Rule	IF condition	THEN condition
R1	Attendance < 75%	Attendance concern
R2	Average grade < 75	Academic concern
R3	Both concerns	Priority advising
R4	Priority advising	Recommend adviser consultation
R5	Grade >= 90 AND attendance >= 90%	Recognition review

Tasks

- Represent student records as facts and define at least 8 IF-THEN rules
- Implement forward chaining and allow inferred facts to trigger additional rules
- Prevent duplicate conclusions and show the rules fired for each profile
- Test at least 5 Fictional profiles including boundary values of 75% attendance and grade 75
- Display initial facts, inferred conclusions, and recommendations through simple interface
- Explain facts versus inferences, two limitations, and one improvement

---

- Represent student records as facts and define at least 8 IF-THEN rules

STUDENT RECORDS

```
students <- data.frame(
```

```
  Name = character(),
  Attendance = numeric(),
  Grade = numeric(),
  stringsAsFactors = FALSE
)
```

KNOWLEDGE BASE (8 IF-THEN)

```
rules <- list(
  R1 = function(a, g) {
    if (a < 75) "Attendance concern" else NULL
  },
  R2 = function(a, g) {
    if (g < 75) "Academic concern" else NULL
  },
  R3 = function(a, g) {
    if (a < 75 && g < 75) "Priority advising" else NULL
  },
  R4 = function(a, g) {
    if (a < 75 || g < 75)
      "Recommend adviser consultation" else NULL
  },
  R5 = function(a, g) {
    if (g >= 90 && a >= 90)
      "Recognition review" else NULL
  },
  R6 = function(a, g) {
    if (a >= 75 && g < 75)
      "Recommend academic tutoring" else NULL
  },
  R7 = function(a, g) {
    if (a < 75 && g >= 75)
      "Recommend attendance improvement plan" else NULL
  },
  R8 = function(a, g) {
    if (a >= 75 && g >= 75 && g < 90)
      "Satisfactory academic standing" else NULL
  }
)
```

- Implement forward chaining and allow inferred facts to trigger additional rules

FORWARD CHAINING (This checks the initial facts against the rules and adds any conclusions that are triggered, without adding duplicate conclusions.)

```
forward_chaining <- function(attendance, grade) {
  facts <- c(
    paste("Attendance =", attendance, "%"),
    paste("Grade =", grade)
  )

  conclusions <- character()
  trace <- character()

  repeat {
    previous_count <- length(conclusions)

    for (rule_name in names(rules)) {
      result <- rules[[rule_name]](attendance, grade)

      if (!is.null(result) &&
          !(result %in% conclusions)) {
        conclusions <- c(conclusions, result)
        trace <- c(
          trace,
          paste(rule_name, "fired:", result)
        )
      }
    }

    if (length(conclusions) == previous_count) {
      break
    }
  }

  list(
    facts = facts,
    conclusions = conclusions,
    trace = trace
```

```
  )
}
```

- Prevent duplicate conclusions and show the rules fired for each profile

ADD STUDENT AND PREVENT DUPLICATES

```
add_student <- function(name, attendance, grade) {
  if (name %in% students$Name) {
    cat("ERROR: Duplicate student record!\n")
    return(invisible(FALSE))
  }

  if (attendance < 0 || attendance > 100 ||
      grade < 0 || grade > 100) {
    cat("ERROR: Attendance and grade must be 0-100.\n")
    return(invisible(FALSE))
  }

  students <<- rbind(
    students,
    data.frame(
      Name = name,
      Attendance = attendance,
      Grade = grade,
      stringsAsFactors = FALSE
    )
  )

  cat("Student added:", name, "\n")
  invisible(TRUE)
}
```

- Test at least 5 Fictional profiles including boundary values of 75% attendance and grade 75

```
add_student("Student A", 60, 65)
add_student("Student B", 80, 70)
add_student("Student C", 70, 85)
```

```
add_student("Student D", 95, 95)
add_student("Student E", 85, 82)
4

# Test duplicate prevention
add_student("Student A", 60, 65)

# Generate reports for all five students
for (name in students$Name) {
  advise_student(name)
}

# Start the interactive interface afterward
run_interface()
```

- Display initial facts, inferred conclusions, and recommendations through simple interface

```
================================
 STUDENT ACADEMIC ADVISING REPORT
================================
Student: Student A

INITIAL FACTS:
- Attendance: 60 %
- Grade: 65

INFERRED CONCLUSIONS:
- Attendance concern
- Academic concern
- Priority advising
- Recommend adviser consultation

REASONING TRACE:
- R1 fired: Attendance concern
- R2 fired: Academic concern
- R3 fired: Priority advising
- R4 fired: Recommend adviser consultation
================================

================================
 STUDENT ACADEMIC ADVISING REPORT
================================
Student: Student B
```

```
INITIAL FACTS:
- Attendance: 80 %
- Grade: 70

INFERRED CONCLUSIONS:
- Academic concern
- Recommend adviser consultation
- Recommend academic tutoring

REASONING TRACE:
- R2 fired: Academic concern
- R4 fired: Recommend adviser consultation
- R6 fired: Recommend academic tutoring
================================

================================
 STUDENT ACADEMIC ADVISING REPORT
================================
Student: Student C

INITIAL FACTS:
- Attendance: 70 %
- Grade: 85

INFERRED CONCLUSIONS:
- Attendance concern
- Recommend adviser consultation
- Recommend attendance improvement plan

REASONING TRACE:
- R1 fired: Attendance concern
- R4 fired: Recommend adviser consultation
- R7 fired: Recommend attendance improvement plan
================================

================================
 STUDENT ACADEMIC ADVISING REPORT
================================
Student: Student D

INITIAL FACTS:
- Attendance: 95 %
- Grade: 95

INFERRED CONCLUSIONS:
- Recognition review
```

```
REASONING TRACE:
- R5 fired: Recognition review
================================

================================
 STUDENT ACADEMIC ADVISING REPORT
================================
Student: Student E

INITIAL FACTS:
- Attendance: 85 %
- Grade: 82

INFERRED CONCLUSIONS:
- Satisfactory academic standing

REASONING TRACE:
- R8 fired: Satisfactory academic standing
================================
```

- Explain facts versus inferences, two limitations, and one improvement

1. Facts vs. Inferences

Facts are the actual information entered into the system about a student. In this program, the facts are the student's attendance percentage and grade.

Example:

- Attendance: 60%
- Grade: 65

Inferences are the conclusions the system derives from the facts by applying its IF–THEN rules.

The system infers:

- Attendance concern
- Academic concern
- Priority advising
- Recommend adviser consultation

2. Two Limitations

1. Limited knowledge base. The system only uses eight predefined rules. It cannot handle situations that are not covered by those rules.

2. No permanent data storage. Student records are stored in memory. When the program closes, the records are lost.

3. One Improvement

Add a database. Integrating MySQL would allow the system to save student records permanently, retrieve previous records, and generate advising reports even after the program closes.

ADDED FEATURES:

USER INTERFACE:

```
run_interface <- function() {

  repeat {

    cat("\nSTUDENT ACADEMIC ADVISING SYSTEM\n")

    cat("1. Add student\n")

    cat("2. View all students\n")

    cat("3. Generate advising report\n")

    cat("4. Exit\n")



    choice <- readline("Choose an option: ")



    if (choice == "1") {

      name <- readline("Student name: ")

      attendance <- as.numeric(

        readline("Attendance percentage: ")

      )

      grade <- as.numeric(
```

```
      readline("Grade: ")
    )

    if (is.na(attendance) || is.na(grade)) {
      cat("ERROR: Enter valid numeric values.\n")
    } else {
      add_student(name, attendance, grade)
    }

  } else if (choice == "2") {
    if (nrow(students) == 0) {
      cat("No student records available.\n")
    } else {
      print(students)
    }

  } else if (choice == "3") {
    name <- readline("Enter student name: ")
    advise_student(name)

  } else if (choice == "4") {
    cat("Exiting Academic Advising System.\n")
    break
```

```
    } else {
      cat("Invalid option. Try again.\n")
    }
  }
}
```

STUDENT ADVISING REPORT

```
advise_student <- function(name) {
  student <- students[students$Name == name, ]

  if (nrow(student) == 0) {
    cat("Student not found.\n")
    return(invisible(NULL))
  }

  result <- forward_chaining(
    student$Attendance,
    student$Grade
  )

  cat("\n================================\n")
```

```
cat(" STUDENT ACADEMIC ADVISING REPORT\n")

cat("================================\n")



cat("Student:", student$Name, "\n")



cat("\nINITIAL FACTS:\n")

cat("- Attendance:", student$Attendance, "%\n")

cat("- Grade:", student$Grade, "\n")



cat("\nINFERRED CONCLUSIONS:\n")



if (length(result$conclusions) == 0) {

  cat("- No advising recommendation triggered.\n")

} else {

  for (item in result$conclusions) {

    cat("-", item, "\n")

  }

}



cat("\nREASONING TRACE:\n")



if (length(result$trace) == 0) {

  cat("- No rules fired.\n")
```

```
  } else {

    for (item in result$trace) {

      cat("-", item, "\n")

    }

  }



  cat("================================\n")

}
```

The Student Academic Advising KRR System is a knowledge-based system developed in R that evaluates students' attendance and grades to identify academic concerns and provide advising recommendations.

How the System Works

When the program runs, it displays a menu with four options. The user enters a number to select the desired function.

1. Add Student – Allows the user to enter a student's name, attendance percentage, and grade. The system validates the values and prevents duplicate student records.

2. View All Students – Displays all student records currently stored in the system, including their names, attendance percentages, and grades.

3. Generate Advising Report – Finds the selected student's record, evaluates the facts using eight predefined IF–THEN rules, and generates a report containing the student's information, inferred conclusions, and reasoning trace showing which rules were triggered.

4. Exit – Terminates the interactive program.